

One-Stop Resource For Aspiring Software Engineers

Hey there, I'm Karan! 🙋

It is so awesome to see you here. Thank you for purchasing this guide.

Before we get into it, let me tell you a story about myself:

7 years ago, I failed my 1st JEE attempt with a rank of 2.5 Lakh. I knew I wouldn't get into top colleges.

It was a difficult time & I had no idea what I would do next....

Today, I'm working at Google.

Back in 10th class, I had no idea that the JEE even existed.

I had no idea about the competition and the world waiting for me.

After failing my first attempt, I thought about dropping a year but I didn't

(I'm quite glad now that I didn't waste my time)

And so my journey began at GL Bajaj College in B.Tech 2017.

I was excited, but it all faded as I found the subjects quite dull & boring.

But I enjoy solving problems for fun and I have liked computers since school.

My college lacked a coding culture, but I still carved out my space.

In the next 3 years:

- I prepared for ICPC (International Collegiate Programming Contest)
- went all in with coding, convinced my dept. Head to help me
- started with on Codeforces & CodeChef

By my fifth semester, I was competing in the ICPC Gwalior Regions and

By the sixth, I had reached the Candidate Master level on Codeforces, with a rating of 2021, which placed me within the top 250.

Despite my CP success, companies constantly rejected me during COVID-19.

I learned from my mistakes, and In 2021, I cracked an internship at BrightMoney.

From there, I went to PhonePe, and finally, in 2022, I joined Google.

Last year in 2023, I moved to Google London.

Your background doesn't determine how far you can go.

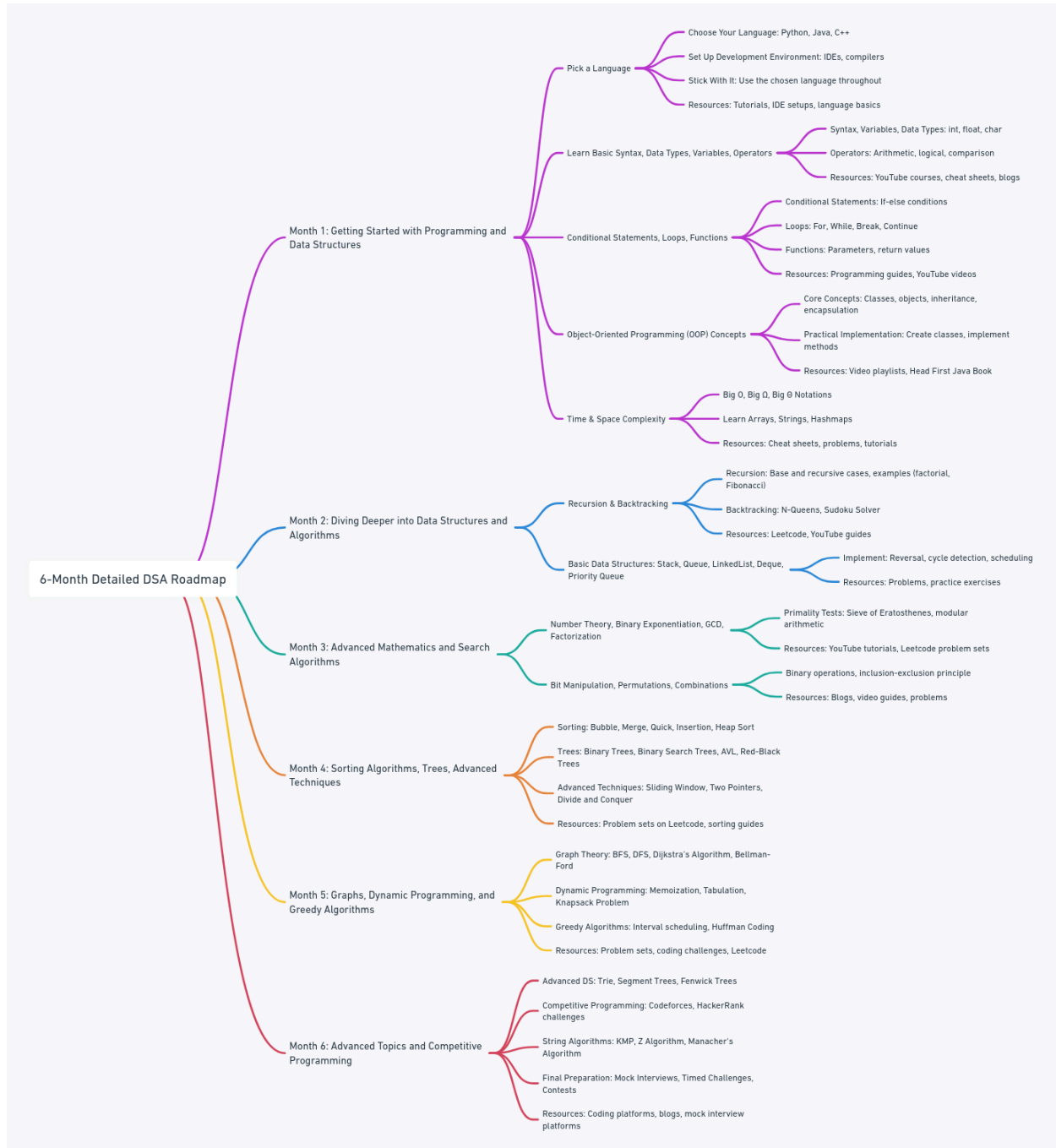
You do.

No matter what you have or where you are, if you work hard, you can make it.

And with that in mind, I have made a one-stop collection of resources.

Part -1: DSA Roadmap

(Click the diagram below to open and zoom into it)



Month 1: Getting Started with Programming and Data Structures

The goal is to lay a solid foundation in programming and data structures.

You'll start by picking a programming language that suits your style and then exploring its core features, such as syntax, control structures, and functions.

You'll also learn essential concepts such as Object-Oriented Programming (OOP) and explore key data structures like arrays, strings, and hashmaps.

By the end of the month, you should be comfortable writing simple programs and understand the basics as well.

► Pick a Language

1. **Choose Your Language:** Choose a language (e.g., Python, Java, C++) that suits your needs and interests.
2. **Set Up Development Environment:** Install necessary tools such as IDEs and compilers.
3. **Stick With It:** Plan to use this language consistently throughout your journey.
4. **Resources:**
 - a. [Get an idea about programming](#)
 - b. [Pick a language and start learning](#)
 - c. Setting up the basics
 - i. [IDE](#)
 - ii. [C++ Setup](#)
 - iii. [Java Setup](#)
 - iv. [Python Setup](#)

► Learn Basic Syntax, Data Types, Variables, Operators

1. **Syntax Basics:** Start with writing simple programs and basic mathematical operations.
2. **Data Types and Variables:** Understand primitive data types (int, float, char) and how to declare variables.
3. **Operators:** Learn different operators (arithmetic, logical, comparison) and how to use them in expressions.
4. **Resources:**
 - a. Basic Fundamentals For Each Language
 - i. [C++](#)

- 1. [Youtube Course](#)
 - ii. [Python](#)
 - 1. [Youtube Course](#)
 - iii. [Java](#)
 - 1. [Youtube Course](#)
- b. Cheatsheets to remember everything
 - i. [Python Cheat Sheet](#)
 - ii. [CPP Cheat Sheet](#)
 - iii. [Java Cheat Sheet](#)

► Conditional Statements, Loops, and Functions

1. **Conditional Statements:** Practice using if-else conditions to make decisions in your programs.
2. **Loops:** Understand and implement loops (for, while) to repeat actions. Learn the use of break and continue.
3. **Functions:** Define and call functions, understand parameters and return values, and write reusable code blocks.
4. **Resources:**
 - a. Courses for live examples of basics in action
 - i. [Full Course C++](#)
 - ii. [Full Course Python](#)
 - iii. [Full Course Java](#)
 - b. [Conditional Statements](#)
 - i. [Java](#)
 - ii. [Python](#)
 - iii. [CPP](#)
 - c. [Loops](#)
 - i. C++
 1. [While Loops](#)
 2. [For Loops](#)
 - ii. Java
 1. [While Loops](#)
 2. [For Loops](#)
 - iii. Python
 1. [While Loops](#)
 2. [For Loops](#)
 - d. [Functions](#)
 - i. [C++](#)
 - ii. [Java](#)
 - iii. [Python](#)

► Understand Object-Oriented Programming (OOP) Concepts

1. **Core Concepts:** Learn about classes, objects, inheritance, polymorphism, encapsulation, and abstraction.
2. **Practical Implementation:** Write simple OOP programs, like creating classes and implementing methods for basic operations.
3. **Resources:**
 - a. Java
 - i. Video Playlist - [OOPS in Java](#)
 - ii. [Reading Material](#)
 - iii. [Head First Java Book](#)
 - b. CPP
 - i. Video Playlist - [OOPS CPP](#)
 - ii. [Reading Material](#)
 - c. Python
 - i. Video Playlist- [OOPS Python](#)
 - ii. [Reading Material](#)

► Learn About Time & Space Complexity

1. **Basics:** Understand the importance of time complexity in evaluating algorithm efficiency.
2. **Resources:**
 - a. [Big O - Cheatsheet](#)
 - b. [Youtube Video](#)

► Big O, Big Ω , Big Θ Notations

1. **Definitions:** Understand the definitions of Big O (upper bound), Big Ω (lower bound), and Big Θ (tight bound) notations.
2. **Applications:** Apply these notations to describe the efficiency of algorithms.
3. **Resources:**
 - a. [Youtube Video](#)
 - b. [Reading Material](#)

► Learn About Arrays, Strings, Hashmaps

1. **Arrays:** Understand how to declare, initialize, and access array elements. Practice basic operations like traversing and modifying array elements.
 - a. Resources:
 - i. [Playlist](#)
 - ii. [Reading Material](#)
 - iii. **Problems:**
 1. [Basic to Advanced Array Questions with Techniques](#)
 2. [Easy Questions List - Leetcode](#)

2. **Strings:** Learn string manipulation techniques, including concatenation, slicing, and searching.
 - a. Resources:
 - i. [Youtube Video](#)
 - ii. [Reading Material](#)
 - iii. **Problems:**
 1. [Types of String Problems](#)
 2. [Easy Questions List - Leetcode](#)
3. **Hashmaps:** Introduction to hashmaps, understanding key-value pairs, and basic operations like insertion, deletion, and lookup.
 - a. Resources:
 - i. [Youtube Videos](#)
 - ii. [Reading Material](#)
 - iii. Problems
 1. Hashing Guide
 - a. [Hashing Guide 1 Leetcode](#)
 2. [Easy Questions List - Leetcode](#)

Do These Before You Enter Month 2

► Step-by-Step LeetCode Guide (You'll be Using It More and More)

Step 1: Set Up Your LeetCode Environment

- **Home Base:** Use the [LeetCode Algorithms Problem Set](#) as your starting point.
- **Sort Problems:** Click the "Difficulty" header twice to sort from easiest to hardest. This also sorts them within each difficulty level by reverse acceptance rate.
- **Unsolved Problems on Top:** Click the empty header in the first column to sort unsolved problems to the top.

Step 2: Begin Solving Problems

- **Start Easy:** Begin with the easiest problems and progress to harder ones. If problems become too easy, start skipping some and move to medium problems.
- **Use Editorials:** Start with problems that have an editorial (indicated by a document icon in the "Solution" column).
- **High-Quality Problems:** Focus on problems with good reviews. Avoid problems with a worse than 2:1 upvote-to-downvote ratio. Problems with a 4:1 ratio or better are usually high quality.

Step 3: Getting Unstuck

If you find yourself stuck on a problem, here are some steps to help you move forward:

1. **Read the Editorial:** If available, start with the editorial. These are usually high-quality and provide well-written solutions.
2. **Check the Discuss Tab:** Read posts in the "Discuss" tab. Quality varies, but you might find detailed explanations and alternative solutions.
3. **Google the Problem:** Sometimes, searching the problem online can lead to solutions on blogs or GitHub repositories.

Step 4: Refining Your Skills

Once you are comfortable with solving problems on LeetCode, focus on refining your skills:

1. **Filter by Tags:** If you need to strengthen a specific algorithm, filter problems by tags. Note that this is not recommended for general practice, as identifying the type of algorithm is part of problem-solving.
2. **Optimize Solutions:** Revisit older problems to ensure you've found the optimal solution. LeetCode may accept suboptimal solutions, but strive for the best possible runtime and memory efficiency.
3. **Participate in Contests:** Weekly contests on LeetCode are a great way to benchmark your skills against others and introduce time pressure.

Resources:

1. [How to use Leetcode efficiently and effectively by beginners](#)

Month 2: Diving Deeper into Data Structures and Algorithms

This month, you'll deepen your understanding of essential data structures and algorithms. You'll focus on recursion, backtracking, and various fundamental data structures. Additionally, you'll explore key algorithms like Kadane's algorithm and the difference array.

By the end of this month, you'll have a strong grasp of these topics, which are crucial for tackling more complex programming challenges.

► Recursion

1. **What to Focus On:**
 - a. Understanding the principles of recursion.
 - b. Differentiating between base cases and recursive cases.
 - c. Practicing problems like factorial calculation, Fibonacci sequence, and tower of Hanoi.
2. **Resources:**
 - a. [Recursion Basics \(YouTube\)](#)
 - b. [Recursion Guide + Problems - Leetcode Blog](#)
 - c. **Problems:**
 - i. [Reverse Linked List](#)

- ii. [Merge Two Sorted Lists](#)
- iii. [Palindrome Linked List](#)
- iv. [Remove Linked List Elements](#)
- v. [Reverse String](#)
- vi. [Power of Two](#)
- vii. [Fibonacci Number](#)
- viii. [Power of Four](#)
- ix. [Find the Winner of the Circular Game](#)
- x. [Power of Three](#)
- xi. [Add Two Numbers](#)
- xii. [Decode String](#)
- xiii. [Swap Nodes in Pairs](#)
- xiv. [Reorder List](#)
- xv. [Pow\(x, n\)](#)

► Backtracking

- **What to Focus On:**
 - Mastering the backtracking approach for solving problems.
 - Implementing solutions for classic problems like the [N-Queen](#) problem and [Sudoku solver](#).
- **Resources:**
 - [Backtracking Video Course \(YouTube\)](#)
 - **Problems:**
 - i. [Generate Parentheses](#)
 - ii. [Permutations](#)
 - iii. [Combination Sum](#)
 - iv. [Letter Combinations of a Phone Number](#)
 - v. [Subsets](#)
 - vi. [Combinations](#)
 - vii. [All Paths From Source to Target](#)
 - viii. [Word Ladder II](#)
 - ix. [Letter Case Permutation](#)
 - x. [Restore IP Addresses](#)
 - xi. [N-Queens](#)
 - xii. [Sudoku Solver](#)
 - xiii. [Word Search II](#)
 - xiv. [Expression Add Operators](#)
 - xv. [Palindrome Partitioning II](#)

► Basic Data Structures

- **What to Focus On:**

- Strengthening your understanding of basic data structures: Stack, Queue, LinkedList, Deque, and Priority Queue.
- Implementing these structures and solving related problems like reversal and cycle detection.
- **Resources:**
 - [Data Structures Overview \(YouTube\)](#)

► Stack

- **What to Focus On:**
 - a. Understanding stack operations and applications.
 - b. Solving problems related to expression evaluation and function call management.
- **Resources:**
 - a. [Video Tutorial On Stack](#)
 - b. **Problems:**
 - i. [Trapping Rain Water](#)
 - ii. [Valid Parentheses](#)
 - iii. [Largest Rectangle in Histogram](#)
 - iv. [Palindrome Linked List](#)
 - v. [Decode String](#)
 - vi. [Min Stack](#)
 - vii. [Longest Valid Parentheses](#)
 - viii. [Binary Tree Inorder Traversal](#)
 - ix. [Flatten Binary Tree to Linked List](#)
 - x. [Daily Temperatures](#)
 - xi. [Maximal Rectangle](#)
 - xii. [Reorder List](#)
 - xiii. [Next Greater Element II](#)
 - xiv. [Basic Calculator II](#)
 - xv. [Remove Duplicate Letters](#)

► Queue

- **What to Focus On:**
 - Learning queue operations and their use in scheduling tasks and managing buffers.
- **Resources:**
 - [Video Tutorial On Queue](#)
 - **Problems:**
 - i. [Merge k Sorted Lists](#)
 - ii. [Sliding Window Maximum](#)
 - iii. [Kth Largest Element in an Array](#)
 - iv. [Top k Frequent Elements](#)
 - v. [Find Median from Data Stream](#)

- vi. [Task Scheduler](#)
- vii. [Kth Smallest Element in a Sorted Matrix](#)
- viii. [Sort Characters By Frequency](#)
- ix. [Ugly Number II](#)
- x. [The Skyline Problem](#)
- xi. [Network Delay Time](#)
- xii. [Flatten Nested List Iterator](#)
- xiii. [Implement Queue using Stacks](#)
- xiv. [Find K Pairs with Smallest Sums](#)
- xv. [Maximum Sum Circular Subarray](#)
- xvi. [Shortest Subarray with Sum at Least K](#)

► **Linked List**

- **What to Focus On:**
 - Understanding linked list structures and operations like insertion, deletion, and traversal.
 - Solving problems related to list reversal and cycle detection.
- **Resources:**
 - [Video Tutorials on LinkedIn List](#)
 - **Problems:**
 - i. [Add Two Numbers](#)
 - ii. [LRU Cache](#)
 - iii. [Reverse Linked List](#)
 - iv. [Merge k Sorted Lists](#)
 - v. [Merge Two Sorted Lists](#)
 - vi. [Remove Nth Node From End of List](#)
 - vii. [Intersection of Two Linked Lists](#)
 - viii. [Palindrome Linked List](#)
 - ix. [Copy List with Random Pointer](#)
 - x. [Linked List Cycle](#)
 - xi. [Flatten Binary Tree to Linked List](#)
 - xii. [Reverse Nodes in k-Group](#)
 - xiii. [Linked List Cycle II](#)
 - xiv. [Sort List](#)
 - xv. [Swap Nodes in Pairs](#)

► **Reversal In Linked List**

- **What to Focus On:**
 - Implementing algorithms to reverse linked lists and arrays.
 - Understanding the importance of reversal operations in various scenarios.
- **Resources:**
 - [Video Tutorial on Reversing LinkedIn List](#)

- **Problems:**
 - i. [Reverse Bits](#)
 - ii. [Reverse String](#)
 - iii. [Reverse Linked List](#)
 - iv. [Reverse String II](#)
 - v. [Reverse Only Letters](#)
 - vi. [Reverse Prefix of Word](#)
 - vii. [Reverse Vowels of a String](#)
 - viii. [Reverse Words in a String III](#)
 - ix. [Existence of a Substring in a String After Its Reverse](#)
 - x. [A Number After a Double Reversal](#)
 - xi. [Make Two Arrays Equal by Reversing Subarrays](#)
 - xii. [Reverse Integer](#)
 - xiii. [Reverse Linked List II](#)
 - xiv. [Evaluate Reverse Polish Notation](#)
 - xv. [Reverse Words in a String](#)

► Cycle Detection in LinkedIn List

- **What to Focus On:**
 - Learning algorithms to detect cycles in linked lists and graphs.
 - Understanding the significance of cycle detection in avoiding infinite loops.
- **Resources:**
 - [Floyd's cycle detection algorithm](#)
 - [Full Approach Guide - Leetcode](#)
 - **Problems:**
 - i. [Linked List Cycle Detection](#)
 - ii. [Linked List Cycle](#)

► Deque

- **What to Focus On:**
 - Understanding double-ended queues (deque) and their operations.
 - Exploring the use of deques in solving sliding window problems.
- **Resources:**
 - [Deque Data Structure \(YouTube\)](#)
 - **Problems:**
 - i. [Strange Printer](#)
 - ii. [Design Circular Deque](#)

► Priority Queue

- **What to Focus On:**
 - Learning about priority queues and their implementation using heaps.
 - Solving problems where elements need to be processed based on priority.

- **Resources:**

- [Priority Queue Explained \(YouTube\)](#)
- **Problems:**
 - i. [Kth Largest Element in a Stream](#)
 - ii. [Last Stone Weight](#)
 - iii. [Top K Frequent Elements](#)
 - iv. [Merge k Sorted Lists](#)
 - v. [Find K Closest Elements](#)
 - vi. [K Closest Points to Origin](#)
 - vii. [Minimum Cost to Connect Sticks](#)
 - viii. [Task Scheduler](#)
 - ix. [Reorganize String](#)
 - x. [Distant Barcodes](#)
 - xi. [Maximum Frequency Stack](#)
 - xii. [Minimum Number of Refueling Stops](#)
 - xiii. [Super Ugly Number](#)
 - xiv. [Find Median from Data Stream](#)
 - xv. [Swim in Rising Water](#)

► Kadane's Algorithm

- **What to Focus On:**

- Understanding the principles behind Kadane's algorithm for maximum subarray problems.
- Applying Kadane's algorithm to solve related coding challenges.

- **Resources:**

- [Kadane's Algorithm Tutorial \(YouTube\)](#)
- **Problems:**
 - i. [Maximum Subarray Problem](#)
 - ii. [Maximum Difference Of 0's And 1's In A Binary String](#)
 - iii. [Maximum Sum Circular Subarray](#)
 - iv. [Smallest Sum Contiguous Subarray](#)
 - v. [Largest Sum Increasing Contiguous Subarray](#)

► Difference Array

- **What to Focus On:**

- Learning how difference arrays simplify range update queries.
- Implementing difference arrays in interval-based problems.

- **Resources:**

- [Difference Array Explanation \(YouTube\)](#)
- **Problems:**
 - i. [Find the Distinct Difference Array](#)
 - ii. [Difference Between Element Sum and Digit Sum of an Array](#)

- iii. [Find the Difference](#)
- iv. [Minimum Absolute Difference](#)
- v. [Odd String Difference](#)
- vi. [Highest Salaries Difference](#)
- vii. [Find the Difference of Two Arrays](#)
- viii. [Minimum Absolute Difference in BST](#)
- ix. [Maximum Difference Between Increasing Elements](#)
- x. [Divisible and Non-Divisible Sums Difference](#)
- xi. [Permutation Difference Between Two Strings](#)

► Intervals

- **What to Focus On:**
 - a. Studying algorithms that handle interval problems, such as merging intervals and finding overlaps.
 - b. Implementing interval-based solutions in competitive programming.
- **Resources:**
 - a. [Interval Problems \(YouTube\)](#)
 - b. **Problems:**
 - i. [Merge Intervals](#)
 - ii. [Non-overlapping Intervals](#)
 - iii. [Remove Covered Intervals](#)
 - iv. [Intervals Between Identical Elements](#)
 - v. [Partition Array Into Disjoint Intervals](#)
 - vi. [Divide Intervals Into Minimum Number of Groups](#)
 - vii. [Interval List Intersections](#)
 - viii. [Insert Interval](#)
 - ix. [Remove Interval](#)
 - x. [Custom Interval](#)
 - xi. [Find Right Interval](#)
 - xii. [Find Interview Candidates](#)
 - xiii. [Longest Well-Performing Interval](#)
 - xiv. [Interleaving String](#)
 - xv. [Accepted Candidates From the Interviews](#)

Month 3: Advanced Mathematics and Search Algorithms

The goal is to learn the mathematical foundations and advanced number theory concepts essential for problem-solving in competitive programming.

You'll also dive into bit manipulation techniques and explore various search algorithms, including linear, binary, and ternary search.

► Maths and Number Theory

- **What to Focus On:**
 - Strengthening your understanding of essential mathematical concepts used in algorithms.
 - Applying these concepts to solve problems related to number theory, factorization, and divisibility.
- **Resources:**
 - [Number Theory Basics \(YouTube\)](#)
 - [Number Theory - CodeChef Guide](#)
 - **Problems:**
 - i. [Add Digits](#)
 - ii. [Three Divisors](#)
 - iii. [Find Greatest Common Divisor of Array](#)
 - iv. [Smallest Even Multiple](#)
 - v. [Number of Common Factors](#)
 - vi. [Count Primes](#)
 - vii. [Mirror Reflection](#)
 - viii. [Simplified Fractions](#)
 - ix. [The kth Factor of n](#)
 - x. [Number of Subarrays With GCD Equal to K](#)
 - xi. [Largest Component Size by Common Factor](#)
 - xii. [Graph Connectivity With Threshold](#)
 - xiii. [Count Ways to Make Array With Product](#)
 - xiv. [Maximize Score After N Operations](#)
 - xv. [Number of Different Subsequences GCDs](#)

► Binary Exponentiation

- **What to Focus On:**
 - Learning the binary exponentiation technique for fast exponentiation.
 - Implementing binary exponentiation to solve power-related problems efficiently.
- **Resources:**
 - [Binary Exponentiation Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [UVA Problem 12730 - Dystopian Book Club](#)
 - ii. [UVA Problem 374 - Big Mod](#)
 - iii. [UVA Problem 10523 - Very Easy !!!](#)
 - iv. [Codeforces Problem 630I - Parking Lot](#)
 - v. [LeetCode Problem - Count Good Numbers](#)
 - vi. [CodeChef Problem - RIFFLES](#)
 - vii. [Codeforces Problem 222E - Decoding Genome](#)
 - viii. [Codeforces Problem 852B - Geometric Progression](#)
 - ix. [Codeforces Problem 1117D - Magic Carpet](#)

- x. [SPOJ Problem - LASTDIG](#)
- xi. [SPOJ Problem - LOCKER](#)
- xii. [UVa Live Problem 3722 - Power of Cryptography](#)
- xiii. [SPOJ Problem - ZSUM](#)
- xiv. [Codeforces Problem 498E - Cinema](#)

► GCD (Greatest Common Divisor)

- **What to Focus On:**
 - Understanding the Euclidean algorithm for computing the GCD.
 - Applying the GCD in problems involving fractions, divisibility, and modular arithmetic.
- **Resources:**
 - [GCD Explanation \(YouTube\)](#)
 - **Problems:**
 - i. [GCD and LCM Problem](#)
 - ii. [Find Greatest Common Divisor of Array](#)
 - iii. [GCD Sort of an Array](#)
 - iv. [Count Primes](#)
 - v. [Greatest Common Divisor of Strings](#)
 - vi. [X of a Kind in a Deck of Cards](#)
 - vii. [Largest Component Size by Common Factor](#)
 - viii. [Number of Different Subsequences GCDs](#)
 - ix. [Count the Number of Ideal Arrays](#)
 - x. [Minimum Deletions to Make Array Divisible](#)
 - xi. [Number of Subarrays With GCD Equal to K](#)

► Integer Factorization

- **What to Focus On:**
 - Learning methods for factorizing integers into prime factors.
 - Solving problems related to divisors, factorization, and prime numbers.
- **Resources:**
 - [Integer Factorization Techniques \(YouTube\)](#)
 - **Problems:**
 - i. [Largest Component Size by Common Factor](#)
 - ii. [Distinct Prime Factors of Product of Array](#)
 - iii. [Minimum Factorization](#)
 - iv. [Greatest Common Divisor Traversal](#)
 - v. [Count Primes](#)
 - vi. [Ugly Number](#)
 - vii. [Ugly Number II](#)
 - viii. [Ugly Number III](#)
 - ix. [Largest Component Size by Common Factor](#)

- x. [Number of Distinct Prime Factors of Product of Array](#)
- xi. [Smallest Value After Replacing With Sum of Prime Factors](#)
- xii. [Find Kth Factor of N](#)
- xiii. [Count the Number of Ideal Arrays](#)

► Primality Tests (Linear Sieve, Sieve of Eratosthenes)

- **What to Focus On:**
 - Understanding and implementing primality tests like the Sieve of Eratosthenes.
 - Using these tests to efficiently determine prime numbers up to a given limit.
- **Resources:**
 - [Sieve of Eratosthenes Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Count Primes](#)
 - ii. [Prime Palindrome](#)
 - iii. [Find the Kth Prime](#)
 - iv. [Super Ugly Number](#)
 - v. [Prime Subtraction Operation](#)
 - vi. [Largest Component Size by Common Factor](#)
 - vii. [Find Greatest Common Divisor of Array](#)
 - viii. [Replace Non-Coprime Numbers in Array](#)
 - ix. [Number of Good Ways to Split a String](#)
 - x. [Sum of Beauty of All Substrings](#)

► Sum of Divisors / Number of Divisors

- **What to Focus On:**
 - Calculating the sum and count of divisors for a given number.
 - Applying these concepts in number theory problems.
- **Resources:**
 - [Sum and Count of Divisors \(YouTube\)](#)
 - **Problems:**
 - i. [Count of Smaller Numbers After Self](#)
 - ii. [Count Primes](#)
 - iii. [The Kth Factor of N](#)
 - iv. [Number of Good Ways to Split a String](#)
 - v. [Find Greatest Common Divisor of Array](#)
 - vi. [Three Divisors](#)
 - vii. [Maximum Sum of an Hourglass](#)
 - viii. [Largest Component Size by Common Factor](#)
 - ix. [Sum of Beauty of All Substrings](#)
 - x. [Replace Non-Coprime Numbers in Array](#)

► Permutations and Combinations (Star and Bars, The Inclusion-Exclusion Principle)

- **What to Focus On:**
 - Learning advanced combinatorial techniques like permutations, combinations, and the inclusion-exclusion principle.
 - Applying these techniques in solving combinatorial problems.
- **Resources:**
 - [Permutations and Combinations \(YouTube\)](#)
 - [Inclusion And Exclusion Principle \(YouTube\)](#)
 - [Stars and Bars \(YouTube\)](#)
 - **Problems:**
 - i. [Permutations](#)
 - ii. [Combinations](#)
 - iii. [Letter Combinations of a Phone Number](#)
 - iv. [Subsets](#)
 - v. [Combination Sum](#)
 - vi. [Permutations II](#)
 - vii. [Combination Sum II](#)
 - viii. [Subsets II](#)
 - ix. [Generate Parentheses](#)
 - x. [Factor Combinations](#)
 - xi. [Count Vowels Permutation](#)
 - xii. [Count Numbers with Unique Digits](#)
 - xiii. [Unique Paths III](#)
 - xiv. [Beautiful Arrangement](#)
 - xv. [Reconstruct Itinerary](#)

► Pigeonhole Principle

- **What to Focus On:**
 - Understanding the pigeonhole principle and its applications in proving the existence of certain conditions.
 - Solving problems that involve proving existence using this principle.
- **Resources:**
 - [Pigeonhole Principle Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Majority Element](#)
 - ii. [Find the Duplicate Number](#)
 - iii. [Missing Number](#)
 - iv. [Contains Duplicate](#)
 - v. [Kth Largest Element in an Array](#)
 - vi. [Minimum Window Substring](#)
 - vii. [Longest Substring Without Repeating Characters](#)

- viii. [Subarray Sum Equals K](#)
- ix. [Longest Consecutive Sequence](#)

► Bit Manipulation

- **What to Focus On:**
 - Learning bit manipulation techniques for optimizing operations and solving binary-related problems.
 - Implementing operations like bit masking, bit shifting, and counting bits.
- **Resources:**
 - [Bit Manipulation Basics \(YouTube\)](#)
 - [All Types of Patterns for Bits Manipulations and How to use it \(Blog\)](#)
 - **Problems:**
 - i. [Add Binary](#)
 - ii. [Single Number](#)
 - iii. [Number of 1 Bits](#)
 - iv. [Power of Two](#)
 - v. [Palindrome Permutation](#)
 - vi. [Find the Duplicate Number](#)
 - vii. [Maximum Product of Word Lengths](#)
 - viii. [Hamming Distance](#)
 - ix. [Stickers to Spell Word](#)
 - x. [K-th Symbol in Grammar](#)
 - xi. [Transform to Chessboard](#)
 - xii. [Chalkboard XOR Game](#)
 - xiii. [Shortest Path to Get All Keys](#)
 - xiv. [Triples with Bitwise AND Equal To Zero](#)
 - xv. [Can I Win](#)

► Search Algorithms

◆ Linear Search

- **What to Focus On:**
 - Understanding and implementing the linear search algorithm for unsorted data.
 - Analyzing the time complexity and performance of linear search.
- **Resources:**
 - [Linear Search Explained \(YouTube\)](#)
 - [Linear Search Explained \(Leetcode\)](#)
 - **Problems:**
 - i. [Find Numbers with Even Number of Digits](#)
 - ii. [Find Pivot Index](#)
 - iii. [Check If N and Its Double Exist](#)

- iv. [Maximum Population Year](#)
- v. [Lucky Numbers in a Matrix](#)
- vi. [Find the Difference](#)
- vii. [Find All Numbers Disappeared in an Array](#)
- viii. [Find Common Characters](#)
- ix. [Find Smallest Letter Greater Than Target](#)
- x. [Find the Town Judge](#)
- xi. [Search a 2D Matrix II](#)
- xii. [Find Peak Element](#)
- xiii. [Search in Rotated Sorted Array](#)
- xiv. [First Missing Positive](#)
- xv. [Minimum Size Subarray Sum](#)

◆ **Binary Search**

- **What to Focus On:**
 - Mastering binary search for efficiently finding elements in a sorted array.
 - Implementing binary search and understanding its applications in various problems.
- **Resources:**
 - [Binary Search Tutorial \(YouTube\)](#)
 - [Binary Search Guide - Leetcode](#)
- **Problems:**
 - i. [Merge Two Binary Trees](#)
 - ii. [Cousins in Binary Tree](#)
 - iii. [Evaluate Boolean Binary Tree](#)
 - iv. [Maximum Odd Binary Number](#)
 - v. [Maximum Depth of Binary Tree](#)
 - vi. [Minimum Depth of Binary Tree](#)
 - vii. [Binary Number With Alternating Bits](#)
 - viii. [Binary Prefix Divisible by 5](#)
 - ix. [Average of Levels in Binary Tree](#)
 - x. [Special Positions in a Binary Matrix](#)
 - xi. [Second Minimum Node in a Binary Tree](#)
 - xii. [Sum of Root To Leaf Binary Numbers](#)
 - xiii. [Minimum Changes To Make Alternating Binary String](#)
 - xiv. [Prime Number of Set Bits in Binary Representation](#)
 - xv. [Convert Binary Number in a Linked List to Integer](#)
 - xvi. [Find the Longest Balanced Substring of a Binary String](#)

◆ **Ternary Search**

- **What to Focus On:**
 - Learning the ternary search algorithm for unimodal functions.

- Implementing ternary search and comparing it with binary search.
- **Resources:**
 - [Ternary Search Tutorial \(YouTube\)](#)
 - [Ternary Search Guide](#)
 - **Problems:**
 - i. [Find the Peak Element](#)
 - ii. [Search in Rotated Sorted Array](#)
 - iii. [Find Minimum in Rotated Sorted Array](#)
 - iv. [Maximum Number in Mountain Array](#)
 - v. [Kth Smallest Element in a Sorted Matrix](#)
 - vi. [Find Minimum in Rotated Sorted Array II](#)
 - vii. [Search a 2D Matrix](#)
 - viii. [Find the Smallest Divisor Given a Threshold](#)
 - ix. [Maximum Average Subarray I](#)
 - x. [Search in Rotated Sorted Array II](#)
 - xi. [Minimize Max Distance to Gas Station](#)
 - xii. [Find Peak Element in 2D Array](#)
 - xiii. [Guess Number Higher or Lower II](#)
 - xiv. [Kth Smallest Prime Fraction](#)

Month 4: Mastering Sorting Algorithms, Trees, and Advanced Techniques

Now, The focus will shift to mastering various sorting algorithms, understanding tree data structures, and delving into advanced problem-solving techniques like sliding windows, divide and conquer, and two-pointer methods.

These topics are crucial for efficiently handling data and solving complex algorithmic challenges. By the end of this month, you will have a firm grasp of sorting mechanisms, tree-based data structures, and advanced strategies for optimizing solutions.

► Sort Algorithms

◆ Bubble Sort

- **What to Focus On:**
 - Understanding the basic principles of bubble sort.
 - Implement bubble sorting and analyzing its time complexity.
- **Resources:**
 - [Bubble Sort Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Sort Colors](#)
 - ii. [Sort Array By Parity](#)
 - iii. [Sort Array By Parity II](#)

- iv. [Sort Integers By The Number of 1 Bits](#)
- v. [Sort Characters By Frequency](#)
- vi. [Relative Sort Array](#)
- vii. [Minimum Number of Arrows to Burst Balloons](#)
- viii. [Wiggle Sort II](#)
- ix. [Sort Array By Increasing Frequency](#)
- x. [Sort the Matrix Diagonally](#)
- xi. [Max Chunks To Make Sorted](#)
- xii. [Sort An Array](#)
- xiii. [Custom Sort String](#)
- xiv. [Maximum Product of Two Elements in an Array](#)
- xv. [Sort Integers By Power Value](#)

◆ **Merge Sort**

- **What to Focus On:**
 - Learning the divide and conquer strategy used in merge sort.
 - Implementing merge sort and understanding its efficiency in sorting large datasets.
- **Resources:**
 - [Merge Sort Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Merge Intervals](#)
 - ii. [Sort List](#)
 - iii. [Merge Sorted Array](#)
 - iv. [Interval List Intersections](#)
 - v. [Merge Two Sorted Lists](#)
 - vi. [Kth Smallest Element in a Sorted Matrix](#)
 - vii. [Intersection of Two Arrays II](#)
 - viii. [Count of Smaller Numbers After Self](#)
 - ix. [Smallest Range II](#)
 - x. [Sort Transformed Array](#)
 - xi. [Reverse Pairs](#)
 - xii. [Max Chunks To Make Sorted II](#)
 - xiii. [Interval List Intersections](#)
 - xiv. [Largest Number](#)
 - xv. [Minimum Number of Arrows to Burst Balloons](#)

◆ **Insertion Sort**

- **What to Focus On:**
 - Understanding the logic behind insertion sort and its best-case scenarios.
 - Implementing insertion sort and analyzing its performance on small datasets.
- **Resources:**

- [Insertion Sort Tutorial \(YouTube\)](#)
- **Problems:**
 - i. [Insertion Sort List](#)
 - ii. [Sort Colors](#)
 - iii. [Wiggle Sort II](#)
 - iv. [Pancake Sorting](#)
 - v. [Sort Array By Parity](#)
 - vi. [Sort Array By Parity II](#)
 - vii. [Sort the Matrix Diagonally](#)
 - viii. [Sort Transformed Array](#)
 - ix. [Sort Integers By Power Value](#)
 - x. [Max Chunks To Make Sorted](#)
 - xi. [Max Chunks To Make Sorted II](#)
 - xii. [Longest Word in Dictionary](#)
 - xiii. [Largest Number](#)
 - xiv. [Custom Sort String](#)
 - xv. [Sort the Matrix Diagonally](#)

◆ **Quick Sort**

- **What to Focus On:**
 - Learning the quicksort algorithm and its partitioning technique.
 - Implementing quicksort and understanding its average and worst-case complexities.
- **Resources:**
 - [Quick Sort Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Kth Largest Element in an Array](#)
 - ii. [3Sum](#)
 - iii. [4Sum](#)
 - iv. [Merge Intervals](#)
 - v. [Sort Colors](#)
 - vi. [Group Anagrams](#)
 - vii. [Kth Smallest Element in a Sorted Matrix](#)
 - viii. [Top K Frequent Elements](#)
 - ix. [Find Median from Data Stream](#)
 - x. [Task Scheduler](#)
 - xi. [Largest Number](#)
 - xii. [Sort List](#)
 - xiii. [Queue Reconstruction by Height](#)
 - xiv. [Max Chunks To Make Sorted II](#)
 - xv. [Wiggle Sort II](#)

◆ **Selection Sort**

- **What to Focus On:**
 - Understanding the selection sort process and when to use it.
 - Implementing selection sort and evaluating its efficiency.
- **Resources:**
 - [Selection Sort Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Kth Largest Element in an Array](#)
 - ii. [Kth Smallest Element in a Sorted Matrix](#)
 - iii. [Wiggle Sort II](#)
 - iv. [Top K Frequent Elements](#)
 - v. [Select Sort an Array](#)
 - vi. [Max Chunks To Make Sorted](#)
 - vii. [Max Chunks To Make Sorted II](#)
 - viii. [Wiggle Sort II](#)
 - ix. [Largest Number](#)
 - x. [Kth Smallest Element in a Sorted Matrix](#)
 - xi. [Find Median from Data Stream](#)
 - xii. [Top K Frequent Elements](#)
 - xiii. [Task Scheduler](#)
 - xiv. [Find Median from Data Stream](#)
 - xv. [Task Scheduler](#)

◆ **Heap Sort**

- **What to Focus On:**
 - Learning how heap sort utilizes a binary heap to sort data.
 - Implementing heap sort and comparing its performance with other sorting algorithms.
- **Resources:**
 - [Heap Sort Tutorial \(YouTube\)](#)
 - **Problems:**
 - i. [Kth Largest Element in an Array](#)
 - ii. [Kth Smallest Element in a Sorted Matrix](#)
 - iii. [Find Median from Data Stream](#)
 - iv. [Top K Frequent Elements](#)
 - v. [Task Scheduler](#)
 - vi. [Largest Number](#)
 - vii. [Sort List](#)
 - viii. [Queue Reconstruction by Height](#)
 - ix. [Max Chunks To Make Sorted II](#)
 - x. [Wiggle Sort II](#)
 - xi. [Find Median from Data Stream](#)
 - xii. [Top K Frequent Elements](#)
 - xiii. [Task Scheduler](#)

- xiv. [Find Median from Data Stream](#)
- xv. [Task Scheduler](#)

► Trees

◆ Binary Trees

- **What to Focus On:**
 - Understanding the structure of binary trees and their properties.
 - Implementing basic operations on binary trees, such as insertion, deletion, and traversal.
- **Resources:**
 - [Binary Trees Basics \(YouTube\)](#)
 - [Binary Trees Guide - LeetCode Blog](#)
 - **Problems:**
 - i. [Binary Tree Inorder Traversal](#)
 - ii. [Same Tree](#)
 - iii. [Symmetric Tree](#)
 - iv. [Maximum Depth of Binary Tree](#)
 - v. [Invert Binary Tree](#)
 - vi. [Binary Tree Level Order Traversal](#)
 - vii. [Binary Tree Paths](#)
 - viii. [Construct Binary Tree from Preorder and Inorder Traversal](#)
 - ix. [Kth Smallest Element in a BST](#)
 - x. [Binary Tree Right Side View](#)
 - xi. [Binary Tree Maximum Path Sum](#)
 - xii. [Flatten Binary Tree to Linked List](#)
 - xiii. [Lowest Common Ancestor of a Binary Tree](#)
 - xiv. [Path Sum III](#)
 - xv. [Pseudo-Palindromic Paths in a Binary Tree](#)

◆ Tree Traversals

- **What to Focus On:**
 - Learning different tree traversal techniques: in-order, pre-order, post-order, and level-order.
 - Implementing traversal algorithms and understanding their applications.
- **Resources:**
 - [Tree Traversal Techniques \(YouTube\)](#)
 - **Problems:**
 - i. [Binary Tree Inorder Traversal](#)
 - ii. [Binary Tree Postorder Traversal](#)
 - iii. [Binary Tree Preorder Traversal](#)
 - iv. [N-ary Tree Preorder Traversal](#)

- v. [N-ary Tree Postorder Traversal](#)
- vi. [Construct Binary Tree from Preorder and Inorder Traversal](#)
- vii. [Binary Tree Level Order Traversal](#)
- viii. [Binary Tree Zigzag Level Order Traversal](#)
- ix. [Construct Binary Tree from Inorder and Postorder Traversal](#)
- x. [Construct Binary Search Tree from Preorder Traversal](#)
- xi. [Binary Tree Level Order Traversal II](#)
- xii. [Construct Binary Tree from Preorder and Postorder Traversal](#)
- xiii. [N-ary Tree Level Order Traversal](#)
- xiv. [Flip Binary Tree to Match Preorder Traversal](#)
- xv. [Vertical Order Traversal of a Binary Tree](#)
- xvi. [Recover a Tree from Preorder Traversal](#)

◆ **Binary Search Trees (BST)**

- **What to Focus On:**
 - Learning the properties of binary search trees and how they differ from general binary trees.
 - Implementing BST operations such as search, insert, delete, and traversal.
- **Resources:**
 - [Binary Search Trees Explained \(YouTube\)](#)
 - **Problems:**
 - i. [Unique Binary Search Trees II](#)
 - ii. [Validate Binary Search Tree](#)
 - iii. [Convert Sorted Array to Binary Search Tree](#)
 - iv. [Binary Search Tree Iterator](#)
 - v. [Kth Smallest Element in a BST](#)
 - vi. [Lowest Common Ancestor of a Binary Search Tree](#)
 - vii. [Closest Binary Search Tree Value](#)
 - viii. [Delete Node in a BST](#)
 - ix. [Minimum Absolute Difference in BST](#)
 - x. [Convert BST to Greater Tree](#)
 - xi. [Trim a Binary Search Tree](#)
 - xii. [Search in a Binary Search Tree](#)
 - xiii. [Construct Binary Search Tree from Preorder Traversal](#)
 - xiv. [All Elements in Two Binary Search Trees](#)
 - xv. [Balance a Binary Search Tree](#)

► **Sliding Window**

- **What to Focus On:**
 - Understanding the sliding window technique for solving problems involving subarrays or substrings.

- Implementing sliding window solutions for maximum, minimum, or sum-related problems.
- **Resources:**
 - [Sliding Window Technique \(YouTube\)](#)
 - **Problems:**
 - i. [Longest Substring Without Repeating Characters](#)
 - ii. [Minimum Window Substring](#)
 - iii. [Sliding Window Maximum](#)
 - iv. [Find All Anagrams in a String](#)
 - v. [Minimum Size Subarray Sum](#)
 - vi. [Permutation in String](#)
 - vii. [Max Consecutive Ones III](#)
 - viii. [Longest Repeating Character Replacement](#)
 - ix. [Longest Substring with At Least K Repeating Characters](#)
 - x. [Maximum Length of Repeated Subarray](#)
 - xi. [Subarray Product Less Than K](#)
 - xii. [Subarrays with K Different Integers](#)
 - xiii. [Shortest Subarray with Sum at Least K](#)
 - xiv. [Maximum Points You Can Obtain from Cards](#)
 - xv. [Contains Duplicate III](#)
 - xvi. [Contains Duplicate II](#)

► Divide and Conquer

- **What to Focus On:**
 - Learning the divide and conquer strategy to break down problems into smaller, manageable parts.
- **Resources:**
 - [Divide and Conquer Explained \(YouTube\)](#)
 - **Problems:**
 - i. [Maximum Subarray](#)
 - ii. [Median of Two Sorted Arrays](#)
 - iii. [Merge k Sorted Lists](#)
 - iv. [Kth Largest Element in an Array](#)
 - v. [Majority Element](#)
 - vi. [Construct Binary Tree from Preorder and Inorder Traversal](#)
 - vii. [Top K Frequent Elements](#)
 - viii. [Search a 2D Matrix II](#)
 - ix. [Sort List](#)
 - x. [Convert Sorted Array to Binary Search Tree](#)
 - xi. [Count of Smaller Numbers After Self](#)
 - xii. [K Closest Points to Origin](#)
 - xiii. [Convert Sorted List to Binary Search Tree](#)
 - xiv. [Construct Binary Tree from Inorder and Postorder Traversal](#)

xv. [The Skyline Problem](#)

► Two Pointers

- **What to Focus On:**
 - Understanding the two-pointer technique and its applications in array and string problems.
 - Implementing two-pointer solutions to problems involving pairs, subarrays, and partitions.
- **Resources:**
 - [Two Pointers Technique \(YouTube\)](#)
 - **Problems:**
 - i. [Trapping Rain Water](#)
 - ii. [3Sum](#)
 - iii. [Container With Most Water](#)
 - iv. [Find the Duplicate Number](#)
 - v. [Next Permutation](#)
 - vi. [Sort Colors](#)
 - vii. [Remove Nth Node From End of List](#)
 - viii. [Move Zeroes](#)
 - ix. [Intersection of Two Linked Lists](#)
 - x. [Palindrome Linked List](#)
 - xi. [Rotate Array](#)
 - xii. [Linked List Cycle](#)
 - xiii. [Partition Labels](#)
 - xiv. [Find Median from Data Stream](#)
 - xv. [Linked List Cycle II](#)

Month 5: Exploring Greedy Algorithms, Graphs, and Dynamic Programming

Here we'll explore more advanced algorithmic techniques, focusing on greedy algorithms, graph theory, and dynamic programming (DP).

You'll learn essential graph algorithms for searching, finding the shortest path, and detecting cycles, as well as dynamic programming strategies to optimize problem-solving.

By the end of this month, you'll have a comprehensive understanding of these algorithms, enabling you to solve complex challenges in competitive programming and beyond.

► Greedy Algorithms

- **What to Focus On:**

- Understand the core idea of greedy algorithms, where you make the locally optimal choice at each step with the hope of finding a global optimum.
- Study classic greedy problems like the coin change problem, fractional knapsack problem, and job sequencing problem.
- **Resources:**
 - [Video tutorials on greedy algorithms.](#)
 - **Problems:**
 - i. [Container With Most Water](#)
 - ii. [Jump Game](#)
 - iii. [Jump Game II](#)
 - iv. [Best Time to Buy and Sell Stock II](#)
 - v. [Partition Labels](#)
 - vi. [Task Scheduler](#)
 - vii. [Shortest Unsorted Continuous Subarray](#)
 - viii. [Queue Reconstruction by Height](#)
 - ix. [Gas Station](#)
 - x. [Remove K Digits](#)
 - xi. [Wildcard Matching](#)
 - xii. [Split Array Largest Sum](#)
 - xiii. [Candy](#)
 - xiv. [Minimum Number of Refueling Stops](#)
 - xv. [Smallest Range Covering Elements from K Lists](#)

► Graphs

- **What to Focus On:**
 - Learn about different graph representations such as adjacency matrix and adjacency list.
 - Understand the basic properties and types of graphs including directed, undirected, weighted, and unweighted graphs.
- **Resources:**
 - [Video tutorials on Graphs](#)
 - **Problems:**
 - i. [Course Schedule](#)
 - ii. [Course Schedule II](#)
 - iii. [Minimum Height Trees](#)
 - iv. [Evaluate Division](#)
 - v. [Clone Graph](#)
 - vi. [Number of Provinces](#)
 - vii. [Cheapest Flights Within K Stops](#)
 - viii. [Is Graph Bipartite?](#)
 - ix. [All Paths From Source to Target](#)
 - x. [Critical Connections in a Network](#)
 - xi. [Reconstruct Itinerary](#)

- xii. [Most Stones Removed with Same Row or Column](#)
- xiii. [Find Eventual Safe States](#)
- xiv. [Reorder Routes to Make All Paths Lead to the City Zero](#)
- xv. [Find the City With the Smallest Number of Neighbors at a Threshold Distance](#)

► Search Algorithms in Graphs

◆ ***Breadth First Search (BFS)***

- **What to Focus On:**
 - Master the BFS algorithm and its use in finding the shortest path in unweighted graphs.
 - Apply BFS to solve problems related to graph traversal and connectivity.
- **Resources:**
 - [Video tutorial on BFS.](#)
 - **Problems:**
 - i. [Course Schedule](#)
 - ii. [Cheapest Flights Within K Stops](#)
 - iii. [Reconstruct Itinerary](#)
 - iv. [Network Delay Time](#)
 - v. [All Paths From Source to Target](#)
 - vi. [Find the Town Judge](#)
 - vii. [Keys and Rooms](#)
 - viii. [Number of Operations to Make Network Connected](#)
 - ix. [Find Eventual Safe States](#)
 - x. [Shortest Path Visiting All Nodes](#)
 - xi. [Reorder Routes to Make All Paths Lead to the City Zero](#)
 - xii. [Minimum Number of Vertices to Reach All Nodes](#)
 - xiii. [Shortest Path with Alternating Colors](#)
 - xiv. [Find If Path Exists in Graph](#)
 - xv. [Parallel Courses II](#)

◆ ***Depth First Search (DFS)***

- **What to Focus On:**
 - Learn the DFS algorithm for exploring graph depths.
 - Understand applications of DFS, such as cycle detection and topological sorting.
- **Resources:**
 - [Video tutorial on DFS algorithm](#)
 - **Problems:**
 - i. [Number of Islands](#)
 - ii. [Validate Binary Search Tree](#)
 - iii. [Lowest Common Ancestor of a Binary Tree](#)

- iv. [Course Schedule](#)
- v. [Binary Tree Maximum Path Sum](#)
- vi. [Path Sum III](#)
- vii. [Diameter of Binary Tree](#)
- viii. [Flatten Binary Tree to Linked List](#)
- ix. [Course Schedule II](#)
- x. [Serialize and Deserialize Binary Tree](#)
- xi. [House Robber III](#)
- xii. [Binary Tree Right Side View](#)
- xiii. [Binary Tree Level Order Traversal](#)
- xiv. [Kth Smallest Element in a BST](#)
- xv. [All Nodes Distance K in Binary Tree](#)

► Shortest Path Algorithms

◆ Dijkstra's Algorithm

- **What to Focus On:**
 - Learn Dijkstra's algorithm for finding the shortest paths in weighted graphs with non-negative weights.
 - Implement Dijkstra's algorithm and analyze its efficiency and applications.
- **Resources:**
 - [Video tutorial on Dijkstra's algorithm.](#)
 - **Problems:**
 - i. [Cheapest Flights Within K Stops](#)
 - ii. [Network Delay Time](#)
 - iii. [Path with Maximum Probability](#)
 - iv. [Minimum Cost to Make at Least One Valid Path in a Grid](#)
 - v. [Find the City With the Smallest Number of Neighbors at a Threshold Distance](#)
 - vi. [The Maze II](#)
 - vii. [The Maze III](#)
 - viii. [Path with Minimum Effort](#)
 - ix. [Reachable Nodes in Subdivided Graph](#)
 - x. [Number of Restricted Paths From First to Last Node](#)

◆ Bellman-Ford Algorithm

- **What to Focus On:**
 - Study the Bellman-Ford algorithm, which handles graphs with negative weights and detects negative weight cycles.
 - Compare Bellman-Ford with Dijkstra's algorithm in terms of complexity and use cases.

- **Resources:**

- [Video tutorial on Bellman-Ford algorithm.](#)
- **Problems:**
 - i. [Network Delay Time](#)
 - ii. [Min Cost to Connect All Points](#)
 - iii. [Is Graph Bipartite?](#)
 - iv. [Cheapest Flights Within K Stops](#)
 - v. [Network Delay Time](#)
 - vi. [Find the City With the Smallest Number of Neighbors at a Threshold Distance](#)
 - vii. [Shortest Path with Alternating Colors](#)
 - viii. [All Pairs Shortest Path](#)
 - ix. [Path with Maximum Probability](#)

◆ **Floyd-Warshall Algorithm**

- **What to Focus On:**

- Understand the Floyd-Warshall algorithm for finding shortest paths between all pairs of vertices.
- Implement the algorithm and explore its applications in network routing and distance calculations.

- **Resources:**

- [Video Tutorial On Floyd-Warshall algorithm](#)
- **Problems:**
 - i. [Cheapest Flights Within K Stops](#)
 - ii. [Network Delay Time](#)
 - iii. [Number of Ways to Arrive at Destination](#)
 - iv. [Evaluate Division](#)
 - v. [Find the City With the Smallest Number of Neighbors at a Threshold Distance](#)
 - vi. [Course Schedule IV](#)

► **Minimum Spanning Tree Algorithms**

◆ **Prim's Algorithm**

- **What to Focus On:**

- Learn how Prim's algorithm finds a minimum spanning tree by growing the tree one edge at a time.
- Implement Prim's algorithm and understand its greedy approach.

- **Resources:**

- [Video tutorial on Prim's algorithm](#)
- **Problems:**
 - i. [Minimum Cost to Connect All Points](#)

- ii. [Minimum Cost to Make at Least One Valid Path in a Grid](#)
- iii. [Minimum Number of Vertices to Reach All Nodes](#)
- iv. [Shortest Path with Alternating Colors](#)
- v. [Minimum Path Sum](#)
- vi. [Path with Maximum Minimum Value](#)
- vii. [Minimum Effort Path](#)
- viii. [Shortest Path in a Grid with Obstacles Elimination](#)
- ix. [Network Delay Time](#)
- x. [Shortest Path Visiting All Nodes](#)
- xi. [Prim's Minimum Spanning Tree \(MST\) Algorithm](#)
- xii. [Shortest Path with Alternating Colors](#)
- xiii. [Find the City With the Smallest Number of Neighbors at a Threshold Distance](#)
- xiv. [Minimum Cost to Reach Destination in Time](#)

◆ **Kruskal's Algorithm**

- **What to Focus On:**
 - Study Kruskal's algorithm, which also finds a minimum spanning tree by sorting all edges and picking the smallest ones without forming cycles.
 - Compare Kruskal's and Prim's algorithms in different scenarios.
- **Resources:**
 - [Video tutorial on Kruskal algorithm](#)
 - **Problems:**
 - i. [Connecting Cities With Minimum Cost](#)
 - ii. [Optimize Water Distribution in a Village](#)
 - iii. [Minimum Cost to Repair Edges](#)
 - iv. [Number of Operations to Make Network Connected](#)
 - v. [Union-Find to Detect Cycle in a Graph](#)
 - vi. [Critical Connections in a Network](#)
 - vii. [Find if Path Exists in Graph](#)
 - viii. [Island Connections](#)
 - ix. [Most Stones Removed with Same Row or Column](#)
 - x. [Graph Valid Tree](#)
 - xi. [Accounts Merge](#)
 - xii. [Redundant Connection](#)
 - xiii. [Earliest Moment When Everyone Become Friends](#)
 - xiv. [Remove Max Number of Edges to Keep Graph Fully Traversable](#)

► **Topological Sort**

- **What to Focus On:**
 - Understand the concept of topological sorting in directed acyclic graphs (DAGs).

- Learn how to apply topological sort in scheduling problems and resolving dependencies.
- **Resources:**
 - [Video tutorial on topological sort](#)
 - **Problems:**
 - i. [Course Schedule](#)
 - ii. [Course Schedule II](#)
 - iii. [Minimum Height Trees](#)
 - iv. [Longest Increasing Path in a Matrix](#)
 - v. [Find Eventual Safe States](#)
 - vi. [Course Schedule IV](#)
 - vii. [Number of Ways to Arrive at Destination](#)
 - viii. [Rank Transform of a Matrix](#)
 - ix. [Sort Items by Groups Respecting Dependencies](#)
 - x. [Loud and Rich](#)
 - xi. [Number of Restricted Paths From First to Last Node](#)
 - xii. [Largest Color Value in a Directed Graph](#)
 - xiii. [Strange Printer II](#)
 - xiv. [Find All Possible Recipes From Given Supplies](#)
 - xv. [Parallel Courses III](#)

► Cycle Detection

- **What to Focus On:**
 - Explore algorithms for detecting cycles in both directed and undirected graphs.
 - Understand the implications of cycles in graphs and how they affect algorithms like topological sort.
- **Resources:**
 - [Video tutorial on cycle detection.](#)
 - **Problems:**
 - i. [Detect Cycle in an Undirected Graph Using BFS](#)
 - ii. [Detect Cycle in an Undirected Graph Using DFS](#)
 - iii. [Detect a Cycle in Directed Graph - Topological Sort \(Kahn's Algorithm\)](#)

► Lowest Common Ancestor (LCA)

- **What to Focus On:**
 - Learn about the Lowest Common Ancestor (LCA) problem in trees, which finds the lowest node that is an ancestor of two given nodes.
 - Implement LCA algorithms, particularly in binary trees and binary search trees.
- **Resources:**
 - [Video tutorial on LCA.](#)
 - **Problems:**
 - i. [Lowest Common Ancestor of a Binary Tree](#)

- ii. [Lowest Common Ancestor of Deepest Leaves](#)
- iii. [Lowest Common Ancestor of a Binary Tree III](#)

► Dynamic Programming (DP)

[Full DP Playlist](#)

List of youtube Videos concept wise, use them if you're stuck on a topic and unable to get unblocked.

◆ Memoization

- **What to Focus On:**
 - Understand the concept of memoization, where you store the results of expensive function calls and reuse them when the same inputs occur again.
 - Apply memoization to optimize recursive algorithms.
- **Resources:**
 - [Video tutorial on memoization.](#)
 - **Problems:**
 - i. [Climbing Stairs](#)
 - ii. [Word Break](#)
 - iii. [Longest Increasing Path in a Matrix](#)
 - iv. [Partition to K Equal Sum Subsets](#)
 - v. [Word Break II](#)
 - vi. [Different Ways to Add Parentheses](#)
 - vii. [Fibonacci Number](#)
 - viii. [All Possible Full Binary Trees](#)
 - ix. [Can I Win](#)
 - x. [Remove Boxes](#)
 - xi. [N-th Tribonacci Number](#)
 - xii. [Shopping Offers](#)
 - xiii. [Sort Integers by The Power Value](#)
 - xiv. [Integer Replacement](#)
 - xv. [Minimum Number of Days to Eat N Oranges](#)

◆ Tabulation

- **What to Focus On:**
 - Learn about tabulation, a bottom-up dynamic programming technique where you solve subproblems first and use their results to build up to the solution of the overall problem.
- **Resources:**
 - [Video tutorial on tabulation](#)
 - **Problems:**
 - i. [Longest Increasing Subsequence](#)
 - ii. [Largest Divisible Subset](#)

- iii. [Russian Doll Envelopes](#)
- iv. [Maximum Length of Pair Chain](#)
- v. [Number of Longest Increasing Subsequence](#)
- vi. [Delete and Earn](#)
- vii. [Longest String Chain](#)
- viii. [Partition Equal Subset Sum](#)
- ix. [Last Stone Weight II](#)
- x. [Partition to K Equal Sum Subsets](#)
- xi. [Longest Common Subsequence](#)
- xii. [Edit Distance](#)
- xiii. [Distinct Subsequences](#)
- xiv. [Minimum ASCII Delete Sum for Two Strings](#)
- xv. [Palindrome Partitioning II](#)

◆ **DP Problem List Topic Wise**

- 1. [Linear Dp](#)
- 2. [String and Dp](#)
- 3. [Dp with Tree and Graph](#)
- 4. [Knapsack based Dp](#)
- 5. [Dp with bits manipulation](#)
- 6. [Dp on math problems](#)
- 7. [Classical dp problems](#)
- 8. [Grid based dp](#)
- 9. [Multidimensional Dp](#)
- 10. [Digit problems with dp](#)
- 11. [Interval problems with dp](#)

Month 6: Advanced String Algorithms and Data Structures

Here, you will explore advanced string algorithms and specialized data structures essential for efficient problem-solving in competitive programming. This month's focus includes mastering pattern matching, string hashing, and various tree-based data structures like Fenwick trees and segment trees. By the end of this month, you'll be well-equipped to handle complex problems involving strings and data structures, setting the stage for more advanced topics in the following months.

► **Strings**

◆ **KMP (Knuth-Morris-Pratt) Algorithm**

- **What to Focus On:**

- Learn the KMP algorithm, which efficiently finds occurrences of a pattern within a text.
- Understand the preprocessing step that creates the longest prefix suffix (LPS) array, which is key to KMP's efficiency.
- **Resources:**
 - [Video tutorial on KMP algorithm.](#)
 - **Problems:**
 - i. [Implement strStr\(\)](#)
 - ii. [Repeated Substring Pattern](#)
 - iii. [Longest Substring Without Repeating Characters](#)
 - iv. [Longest Palindromic Substring](#)
 - v. [Find the Index of the First Occurrence in a String](#)
 - vi. [Find All Anagrams in a String](#)
 - vii. [Minimum Window Substring](#)
 - viii. [Substring with Concatenation of All Words](#)
 - ix. [Zigzag Conversion](#)
 - x. [Longest Common Prefix](#)
 - xi. [Distinct Subsequences](#)
 - xii. [Edit Distance](#)
 - xiii. [Wildcard Matching](#)
 - xiv. [Longest Repeating Substring](#)
 - xv. [Word Search](#)

◆ **String Hashing**

- **What to Focus On:**
 - Understand the concept of string hashing and how it can be used for fast string comparison.
 - Learn about hash functions, hash collisions, and how to mitigate them.
- **Resources:**
 - [Video tutorial on string hashing.](#)
 - **Problems:**
 - i. [Repeated DNA Sequences](#)
 - ii. [Longest Duplicate Substring](#)
 - iii. [Distinct Echo Substrings](#)
 - iv. [Longest Repeating Substring](#)
 - v. [Find Substring With Given Hash Value](#)
 - vi. [Check If a String Contains All Binary Codes of Size K](#)
 - vii. [Maximum Length of Repeated Subarray](#)
 - viii. [Longest Happy Prefix](#)
 - ix. [Number of Distinct Substrings in a String](#)
 - x. [Longest Chunked Palindrome Decomposition](#)
 - xi. [Number of Distinct Islands](#)
 - xii. [Number of Distinct Islands II](#)

- xiii. [Shortest Palindrome](#)
- xiv. [Encode and Decode TinyURL](#)
- xv. [Subtree of Another Tree](#)

◆ **Z Function**

- **What to Focus On:**
 - Learn the Z-function, which is used to calculate the Z-array of a string, representing the length of the longest substring starting from each position that matches the prefix.
- **Resources:**
 - [Video tutorial on Z-function computation](#)
 - **Problems:**
 - i. [Longest Duplicate Substring](#)
 - ii. [Shortest Palindrome](#)
 - iii. [Longest Happy Prefix](#)
 - iv. [Find All Anagrams in a String](#)
 - v. [Substring with Concatenation of All Words](#)
 - vi. [Minimum Window Substring](#)
 - vii. [Longest Palindromic Substring](#)
 - viii. [Palindrome Partitioning II](#)
 - ix. [Longest Repeating Character Replacement](#)
 - x. [Wildcard Matching](#)
 - xi. [Regular Expression Matching](#)
 - xii. [Longest Common Prefix](#)
 - xiii. [Edit Distance](#)
 - xiv. [Word Search II](#)
 - xv. [Interleaving String](#)

► **Disjoint Set Union (DSU)**

- **What to Focus On:**
 - Understand the disjoint set union (DSU) or union-find data structure, which supports union and find operations efficiently.
 - Learn about path compression and union by rank techniques to optimize DSU operations.
- **Resources:**
 - [Video tutorial on DSU.](#)
 - **Problems:**
 - i. [Longest Consecutive Sequence](#)
 - ii. [Graph Valid Tree](#)
 - iii. [Satisfiability of Equality Equations](#)
 - iv. [Checking Existence of Edge Length Limited Paths](#)
 - v. [Smallest String with Swaps](#)

- vi. [Lexicographically Smallest Equivalent String](#)
- vii. [Number of Provinces](#)
- viii. [Number of Operations to Make Network Connected](#)
- ix. [Remove Max Number of Edges to Keep Graph Fully Traversable](#)
- x. [Synonymous Sentences](#)
- xi. [Number of Islands II](#)
- xii. [The Earliest Moment When Everyone Become Friends](#)
- xiii. [Minimize Malware Spread](#)
- xiv. [Minimize Malware Spread II](#)
- xv. [Number of Distinct Islands II](#)

► Tries

- **What to Focus On:**
 - Learn about tries, a tree-like data structure that stores a dynamic set of strings, where keys are usually strings.
 - Understand trie operations like insertion, search, and deletion and how they are used in problems like autocomplete and spell checking.
- **Resources:**
 - [Video tutorial on Tries](#)
 - **Problems:**
 - i. [Word Break](#)
 - ii. [Implement Trie \(Prefix Tree\)](#)
 - iii. [Word Search II](#)
 - iv. [Word Break II](#)
 - v. [Top K Frequent Words](#)
 - vi. [Design Add and Search Words Data Structure](#)
 - vii. [Maximum XOR of Two Numbers in an Array](#)
 - viii. [Number of Matching Subsequences](#)
 - ix. [Palindrome Pairs](#)
 - x. [Search Suggestions System](#)
 - xi. [Concatenated Words](#)
 - xii. [Stream of Characters](#)
 - xiii. [Replace Words](#)
 - xiv. [Longest Word in Dictionary](#)
 - xv. [Map Sum Pairs](#)

► Binary Indexed Tree (BIT) / Fenwick Tree

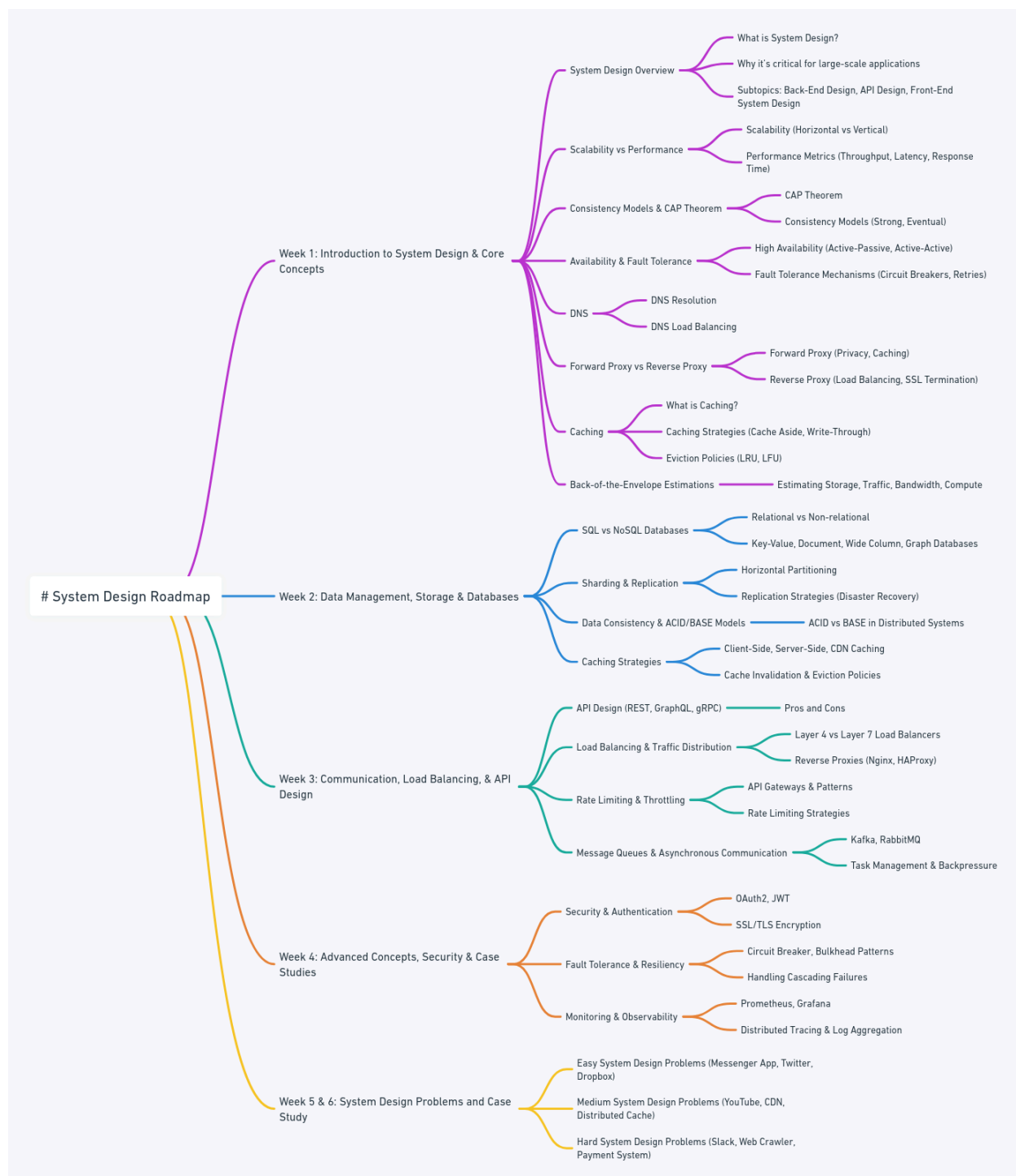
- **What to Focus On:**
 - Learn about binary indexed trees (BIT), also known as Fenwick trees, which efficiently support prefix sum queries and point updates.
 - Understand the construction and application of BITs in range query problems.
- **Resources:**

- [Video tutorial on BIT/Fenwick tree](#)
- **Problems:**
 - i. [The Skyline Problem](#)
 - ii. [Count of Smaller Numbers After Self](#)
 - iii. [Count of Range Sum](#)
 - iv. [Reverse Pairs](#)
 - v. [Create Sorted Array through Instructions](#)
 - vi. [The Skyline Problem](#)
 - vii. [Range Sum Query - Mutable](#)
 - viii. [Range Sum Query 2D - Mutable](#)
 - ix. [Count of Smaller Numbers After Self](#)
 - x. [Count of Range Sum](#)
 - xi. [Queue Reconstruction by Height](#)
 - xii. [Reverse Pairs](#)
 - xiii. [Count Subarrays with More Ones than Zeros](#)
 - xiv. [Number of Longest Increasing Subsequence](#)
 - xv. [K Empty Slots](#)

► Segment Tree

- **What to Focus On:**
 - Understand segment trees, which are used for answering range queries and updating elements in logarithmic time.
 - Learn how to implement a segment tree for various types of queries, including sum, minimum, and maximum queries.
- **Resources:**
 - [Video tutorial on Segment Tree](#)
 - **Problems:**
 - i. [Count of Smaller Numbers After Self](#)
 - ii. [The Skyline Problem](#)
 - iii. [Number of Longest Increasing Subsequence](#)
 - iv. [Range Sum Query - Mutable](#)
 - v. [Reverse Pairs](#)
 - vi. [My Calendar I](#)
 - vii. [Count of Range Sum](#)
 - viii. [My Calendar II](#)
 - ix. [Range Module](#)
 - x. [Rectangle Area II](#)
 - xi. [My Calendar III](#)
 - xii. [Create Sorted Array through Instructions](#)
 - xiii. [Online Majority Element In Subarray](#)
 - xiv. [Falling Squares](#)
 - xv. [Minimum Possible Integer After at Most K Adjacent Swaps On Digits](#)

Part 2 - System Design



Week 1: Introduction to System Design & Core Concepts

Objective:

Familiarize yourself with key concepts like scalability, performance, consistency, and fault tolerance to lay the foundation for understanding system design. You'll gain a basic understanding of the system components that form the backbone of designing scalable, distributed systems.

1. System Design Overview

- **What is System Design?**
 - Why system design is critical for building and scaling large-scale applications.
- **Sub Topics Of System Design:**
 - Back-End Design (handling databases, distributed systems).
 - API Design (designing APIs, managing requests, and communication).
 - Front-End System Design (designing complex client-side applications).

2. Scalability vs Performance

- **What is Scalability?**
 - Scaling systems horizontally vs vertically (adding more machines vs increasing machine power).
 - Key considerations: Cost, maintainability, and fault tolerance.
- **Performance:**
 - Measuring system performance with metrics like throughput, latency, and response time.
 - Balancing performance and scalability for optimal system behavior.

3. Consistency Models & CAP Theorem

- **CAP Theorem:**
 - Trade-offs between Consistency, Availability, and Partition Tolerance in distributed systems.
 - Understanding the limitations of distributed systems through the CAP Theorem.
- **Consistency Models:**
 - Strong Consistency: Guarantees that all clients see the same data at the same time.
 - Eventual Consistency: Guarantees that, given enough time, all replicas will converge to the same state.

4. Availability & Fault Tolerance

- **High Availability:**
 - Designing systems with redundancy to handle failures.
 - Strategies to increase availability: Active-Passive, Active-Active replication.
- **Fault Tolerance:**
 - Designing failover mechanisms for critical systems.
 - Handling transient and permanent failures with circuit breakers and retry mechanisms.

5. DNS (Domain Name System)

- **What is DNS?**
 - Translating human-readable domain names into IP addresses.
 - How DNS resolution works and why it's important for distributed systems.
- **DNS Load Balancing:**
 - Using DNS for load balancing and distributing traffic across servers.

6. Forward Proxy vs Reverse Proxy

- **Forward Proxy:**
 - Intermediary for clients seeking resources from other servers.
 - Usage: Privacy, caching, and filtering requests.
- **Reverse Proxy:**
 - Intermediary between the client and a server.
 - Used for load balancing, caching, and SSL termination.

7. Caching

- **What is Caching?**
 - Temporarily storing data to reduce access time.
 - Cache locations: Client-side, server-side, and CDN caching.
- **Caching Strategies:**
 - Cache Aside, Write-Through, Write-Behind.
 - Caching eviction policies (LRU, LFU).

8. Back-of-the-Envelope Estimations

- **Estimations in System Design:**
 - Rough estimates to quickly validate if a design can meet scalability requirements.
 - Examples: Estimating storage, traffic load, bandwidth, and computational power.
- **Why it's important:**
 - Helps to prioritize resources, spot bottlenecks, and adjust designs early.

Resources:

- **Blogs on System Design Basics:**

- i. [System Design topics roadmap](#)
- ii. [How should you approach System design](#)
- iii. [Scalability vs Performance](#)
- iv. [Scalability System Design](#)
- v. [Performance in System Design and Architecture](#)
- vi. [Scalability, Availability, Reliability](#)
- vii. [CAP Theorem Explained](#)
- viii. [Consistency Models in System Design](#)
- ix. [Consistent Hashing Algorithm](#)
- x. [Rate Limiting Algorithms Explained](#)
- xi. [API Architecture Best Practices for Designing REST APIs](#)
- xii. [Active-active vs. active-passive high-availability clustering](#)
- xiii. [Fault Tolerance](#)
- xiv. [Working of DNS](#)
- xv. [DNS Explained](#)
- xvi. [Understanding Proxy, Forward Proxy, and Reverse Proxy](#)
- xvii. [Caching For System Design](#)
- xviii. [Back-of-the-Envelope Estimations](#)
- xix. [Back-of-the-Envelope Estimations - Video](#)
- b. Additional Topics:
 - i. [Consensus in Distributed Systems](#)
 - ii. [Gossip Protocol Explained](#)
 - iii. [Disaster Recovery](#)
 - iv. [What is Distributed Tracing?](#)
 - v. [Load Balancing Algorithms Explained](#)
 - vi. [Consistency Patterns](#)
 - vii. [Heartbeats in Distributed Systems](#)
 - viii. [Scalable System Design Patterns](#)
 - ix. [Introduction to Architecting Systems for Scale](#)
 - x. [Numbers Everyone Should Know](#)
 - xi. [Fallacies of Distributed Systems](#)
- [YT Videos on System Design Basics](#)

Week 2: Data Management, Storage & Databases

Objective:

Learn about data storage techniques, choosing between SQL and NoSQL databases, and managing data consistency. You'll explore sharding, replication, and other database strategies essential for handling large volumes of data in distributed systems.

Topics Covered:

1. **SQL vs NoSQL**
 - Understanding when to use relational vs non-relational databases.

- Key-Value Stores, Document Stores, Wide Column, and Graph Databases.
- 2. **Sharding & Replication**
 - Horizontal partitioning of data.
 - Replication strategies for high availability and disaster recovery.
- 3. **Data Consistency & ACID/BASE Models**
 - ACID properties in databases.
 - BASE for eventual consistency in distributed systems.
- 4. **Caching Strategies**
 - Caching layers: Client-side, Server-side, CDN caching.
 - Cache invalidation and eviction policies (e.g., LRU, FIFO).

Resources:

- Blogs on concepts:
 - i. [ACID Properites in Databases](#)
 - ii. [ACID VS BASE](#)
 - iii. [BASE in Databases](#)
 - iv. [SQL VS NoSQL Databases](#)
 - v. [SQL VS NoSQL Blog](#)
 - vi. [Sharding vs Replication](#)
 - vii. [What is DATABASE SHARDING?](#)
 - viii. [What is Database Replication?](#)
 - ix. [15 Types of Databases](#)
 - x. [Active-Active Application Architectures](#)
 - xi. [What is Data Replication?](#)
 - xii. [Database Sharding](#)
 - xiii. [Load Balancing Algorithms Explained](#)
 - xiv. [Consistency Patterns](#)
 - xv. [Replication Strategies and Database Indexes](#)
 - xvi. [Consistency Models & CAP Theorem](#)
 - xvii. [Fault Tolerance](#)
 - xviii. [Consensus in Distributed Systems](#)
 - xix. [System Design: Top 15 Trade-Offs](#)
 - xx. [Distributed Caching](#)
 - xxi. [Rate Limiting Algorithms Explained](#)

Week 3: Communication, Load Balancing, & API Design

Objective:

Explore how systems communicate with one another, design APIs that are scalable, and understand load balancing mechanisms for distributing traffic across multiple servers. You'll also dive into the fundamentals of API Gateways, Rate Limiting, and communication patterns.

Topics Covered:

1. **API Design (REST vs GraphQL vs gRPC)**
 - How to design scalable APIs.
 - REST vs GraphQL vs gRPC: Pros and Cons.
2. **Load Balancing & Traffic Distribution**
 - Layer 4 vs Layer 7 load balancers.
 - How to reverse proxies like Nginx and HAProxy distribute traffic.
3. **Rate Limiting & Throttling**
 - Strategies to control the number of requests.
 - API Gateway patterns and design.
4. **Message Queues & Asynchronous Communication**
 - Using Kafka & RabbitMQ for handling asynchronous tasks.
 - Backpressure and task management in distributed systems.

Resources:

- **For API Design:**
 - i. [Task Management In Distributed Systems](#)
 - ii. [Backpressure in Distributed Systems](#)
 - iii. [Rate Limiting Algorithms Explained](#)
 - iv. [API Architecture Best Practices for Designing REST APIs](#)
 - v. [Load Balancing Algorithms Explained](#)
 - vi. [Message Queues](#)
 - vii. [API Gateway](#)
 - viii. [Proxy Server](#)
 - ix. [Gossip Protocol Explained](#)
 - x. [What is Distributed Tracing?](#)
 - xi. [Websockets](#)
 - xii. [Event-Driven Architecture](#)
 - xiii. [Microservices Architecture](#)
 - xiv. [Kafka Crash Course](#)
 - xv. [Scaling a Database](#)
 - xvi. [System Design: Top 15 Trade-Offs](#)
 - xvii. [Throughput vs Latency](#)
- **YT Videos:**
 - i. [API Design](#)
 - ii. [Load Balancing](#)
 - iii. [Message Queues](#)
 - iv. [Asynchronous Communication](#)
 - v. [REST vs GraphQL vs gRPC: Pros and Cons.](#)
 - vi. [Rate Limiting & Throttling](#)
 - vii. [Kafka Crash Course](#)
 - viii. [RabbitMQ](#)

- ix. [Kafka vs. RabbitMQ - who wins and why?](#)

Week 4: Advanced Concepts, Security & Case Studies

Objective:

In the final week, you'll explore advanced system design topics like fault tolerance, and security. You'll also dive into real-world case studies to learn how large-scale systems such as social networks, file-sharing services, and chat applications are designed.

Topics Covered:

1. **Security & Authentication**
 - Implementing secure authentication systems (OAuth2, JWT).
 - Encryption, SSL/TLS, and protecting APIs.
2. **Fault Tolerance & Resiliency**
 - Implementing Circuit Breaker and Bulkhead patterns for fault-tolerant systems.
 - Handling cascading failures in distributed systems.
3. **Monitoring & Observability**
 - Tools like Prometheus, Grafana for monitoring system health.
 - Distributed tracing and log aggregation.
4. **Consensus & Distributed Systems**
 - Quorum consensus for ensuring consistency across distributed databases.
 - Raft consensus algorithm for leader election and fault-tolerance in distributed systems.

Resources:

- **Blog on Microservices & Security:**
 - i. [Design Patterns for Microservices: Circuit Breaker Pattern](#)
 - ii. [Netflix Microservices Architecture](#)
 - iii. [Introduction to Architecting Systems for Scale](#)
 - iv. [Building Microservices](#)
 - v. [API Architecture Best Practices for Designing REST APIs](#)
 - vi. [OAuth2 and JWT Authentication for Microservices](#)
 - vii. [SSL/TLS Encryption and API Security](#)
 - viii. [What is Fault Tolerance?](#)
 - ix. [Circuit Breaker Patterns for Fault Tolerant Systems](#)
 - x. [Cascading Failures and Resilient Patterns](#)
 - xi. [Handling Failover and Resiliency in Distributed Systems](#)
 - xii. [Monitoring and Observability Tools: Prometheus and Grafana](#)
 - xiii. [Distributed Tracing and Log Aggregation](#)
 - xiv. [System Monitoring Tools and Practices](#)
 - xv. [Best Practices for Monitoring Microservices](#)
 - xvi. [What is Quorum Consensus?](#)

- xvii. [What is Raft Consensus Algorithm?](#)
- xviii. Architecture
 - [Client-Server Architecture](#)
 - [Microservices Architecture](#)
 - [Serverless Architecture](#)
 - [Event-Driven Architecture](#)
 - [Peer-to-Peer \(P2P\) Architecture](#)
- **YT Video:**
 - i. [Microservices](#)
 - ii. [Security](#)
 - iii. [Fault Tolerance & Resiliency](#)
 - iv. [Monitoring & Observability](#)
 - v. [Quorum Consensus](#)
 - vi. [Raft Consensus Algorithm](#)

Week 5 & 6: System Design Problems and Case Study

Objective:

Here, it's time to focus on what you've learned and how to implement it all. These case studies will help you learn how systems are designed at scale. Take your time with each of these problems and focus on the approach rather than the specifics.

► Easy System Design Problems:

1. [Design Messenger App](#)
2. [Design Twitter](#)
3. [Design URL Shortener](#)
4. [Design Dropbox](#)
5. [Design Tinder](#)
6. [Design Google Drive](#)
7. [Design Quora](#)
8. [Design Flickr](#)
9. [Design TikTok](#)
10. [Design Netflix](#)
11. [Design Foursquare](#)
12. [Design Uber](#)
13. [Design Facebook's Newsfeed](#)
14. [Design Yelp](#)
15. [Design WhatsApp](#)
16. [Design OneDrive](#)
17. [Design Google Maps](#)
18. [Design Flipkart](#)
19. [Design Tiny URL](#)

20. [Design BookMyShow](#)
21. [Design Parking Garage](#)
22. [Design Vending Machine](#)
23. [Design Unified Payments Interface \(UPI\)](#)

► Medium System Design Problems:

1. [Design Youtube](#)
2. [Design Ticketmaster](#)
3. [Design Text Storage Service like Pastebin](#)
4. [Design Leaderboard](#)
5. [Design Content Delivery Network \(CDN\)](#)
6. [Design Distributed Key-Value Store](#)
7. [Design Distributed Cache](#)
8. [Design Authentication System](#)
9. [Design Distributed Job Scheduler](#)
10. [Design API Rate Limiter](#)

► Hard System Design Problems:

1. [Design Distributed Locking Service](#)
2. [Design Slack](#)
3. [Design Live Comments](#)
4. [Design Youtube](#)
5. [Design Facebook](#)
6. [Design Netflix](#)
7. [Design Web Crawler](#)
8. [Design API Rate Limiter](#)
9. [Design Payment System](#)
10. [Design Food Delivery App](#)
11. [Design Google Docs](#)
12. [Design Zoom](#)
13. [Design Distributed Web Crawler](#)
14. [Design Code Deployment System](#)
15. [Design Distributed Cloud Storage like S3](#)
16. [Design Analytics Platform](#)

Bonus Resources

► Other Good Resources:

1. [Front End System Design Guidebook by GreatFrontEnd](#) - A free system design guidebook focused on front-end system design, e.g., how to design Facebook's News Feed, Autocomplete components, Image Carousels, and more.

2. [System Design in a Hurry](#) - A free course by ex-Meta and Amazon engineers, covering high-level system design concepts, delivery, and core technologies.
3. [System Design Newsletter](#) - A weekly newsletter on system design. Includes a powerful system design template for free.
4. [ByteByteGo](#) - A course by Alex Xu, author of the System Design Interview books. The course covers system design basics and deep dives into designs like YouTube and Facebook Newsfeed.
5. [Grokking the System Design Interview by Design Gurus](#) - A well-known course for system design interviews. It contains popular system design problems and a glossary of system design basics.
6. [Grokking the Advanced System Design Interview](#) - An advanced version of the previous course for more senior-level positions or specialist roles.
7. [Best of System Design Package by Design Gurus](#) - A bundle that includes both System Design Interview courses at a discounted price with lifetime access.
8. [System Design Interview Course by Exponent](#) - A course with videos of mock interviews and reference materials like APIs and database schema.
9. [Gaurav Sen YouTube - System Design Playlist](#) - A playlist by popular YouTuber Gaurav Sen covering system design topics, explained with simple diagrams and detailed explanations.
10. [System Design Interview GitHub](#) - Contains links to tips, blogs, and resources about system design and engineering blogs from top tech companies.
11. [System Design Cheatsheet](#) - A brief, concise revision resource for system design interviews.
12. [System Design Roadmap](#) - A step-by-step guide with links to articles and videos on essential system design topics.

► Books

- [System Design Interview – An Insider's Guide, Second Edition](#) - A beginner-friendly guide to learn system design with easy-to-understand content.
- [High Scalability Website](#)
- [System Design Collection of Concepts](#)
- [System Design Architecture Diagrams](#)
- [Work@tech Collection of Tech Blogs](#)
- [System Design Collection of Concepts](#)
- [Azure architecture center](#)
- [System design questions collection](#)
- [System Design Best Papers and Talks](#)
- [Company and Individual Blogs](#)
- [Just Another Collection](#)
- [Martin Fowler](#)
- [Baisc Concepts Nicely Explained](#)
- [AWS System Design Concepts Compilation](#)
- [Byte Sized System Design by Alex Xu](#)

► Youtube Channels

1. [Gaurav Sen](#)
2. [Code Karle](#)
3. [InfoQ](#)
4. [Backend Engineering 101](#)
5. [Alex Xu Byte Byte Go](#)
6. [Database Depth CMU](#)
7. [Arpit Bhayani Youtube](#)

► Company Engineering Blogs

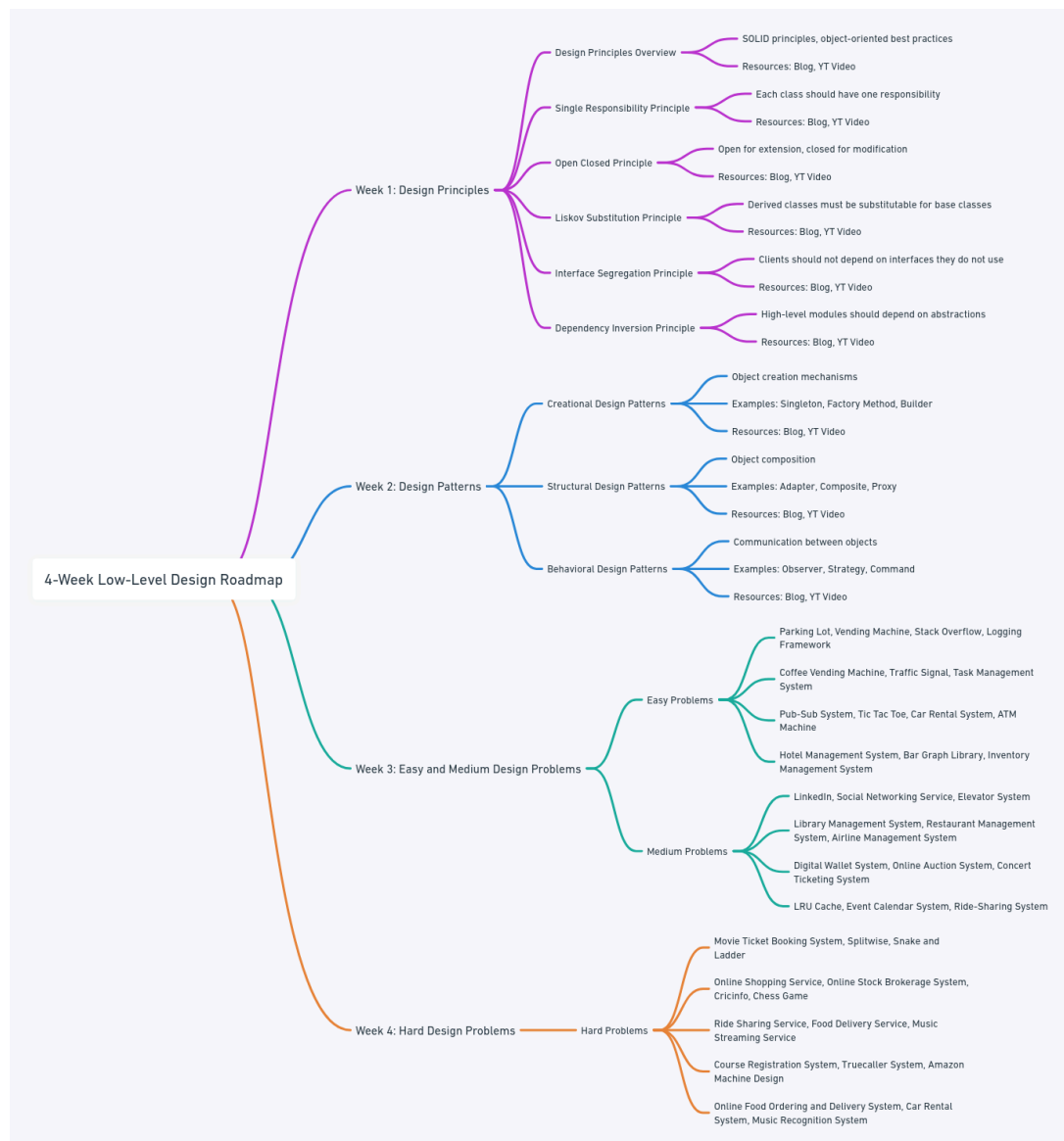
- [High Scalability Blog](#)
- [GitHub Engineering Blog](#)
- [Quora Engineering Blog](#)
- [Yelp Engineering Blog](#)
- [Twitter Engineering](#)
- [Facebook Engineering](#)
- [Airbnb Engineering](#)

Part 3: LLD

One Month Low-Level Design Roadmap

Low-Level Design (LLD) involves creating detailed designs for system components. This includes applying Object-Oriented Programming (OOP), design principles, and design patterns to solve real-world problems.

Understanding these concepts will help you develop scalable, maintainable, and efficient software systems. **(Click the diagram below to open it and zoom to read)**



Week 1: Design Principles

Design principles forms the foundation of creating scalable, maintainable, and efficient systems. This week is dedicated to understanding core design principles such as SOLID, DRY, KISS, and YAGNI. These principles help developers write clean, modular code that is easier to modify and extend.

- **Goal:** Master design principles that ensure code quality and maintainability.
- **Key Focus:** SOLID principles, object-oriented best practices, and general guidelines for robust software design.

► Design Principles Overview

- These principles help in creating clean, modular, and maintainable code.
- Resources:
 - [Blog For Design Principles](#)
 - [YT Video on Design Principles](#)

► Single Responsibility Principle

- **Focus:** Each class should have one responsibility, making code easier to understand, maintain, and modify.
- Resources:
 - [Blog On Single Responsibility Principle](#)
 - [YT Video on Single Responsibility Principle](#)

► Open Closed Principle

- **Focus:** Classes should be open for extension but closed for modification, allowing future flexibility without altering existing code.
- Resources:
 - [Blog On Open Closed Principle](#)
 - [YT Video on Open Closed Principle](#)

► Liskov Substitution Principle

- **Focus:** Derived classes must be substitutable for their base classes without altering the correctness of the program.
- Resources:
 - [Blog On Liskov Substitution Principle](#)
 - [YT Video on Liskov Substitution Principle](#)

► Interface Segregation Principle

- **Focus:** Clients should not be forced to depend on interfaces they do not use.

- Resources:
 - [Blog On Interface Segregation Principle](#)
 - [YT Video on Interface Segregation Principle](#)

Dependency Inversion Principle

- **Focus:** High-level modules should not depend on low-level modules. Both should depend on abstractions.
- Resources:
 - [Blog On Dependency Inversion Principle](#)
 - [YT Video on Dependency Inversion Principle](#)

Week 2: Design Patterns

Design patterns provide standard solutions to common design challenges. This week will focus on recognizing and implementing key design patterns like creational, structural, and behavioral patterns. Understanding when and how to use these patterns will allow you to solve complex problems in an organized manner.

- **Goal:** Learn how to apply design patterns to streamline your coding process.
- **Key Focus:** Creational, structural, and behavioral patterns to design efficient systems.

► Creational Design Patterns

- Focus on object creation mechanisms, trying to create objects in a manner suitable for the situation.
 - **Examples:** Singleton, Factory Method, Builder.
 - Resources:
 - [Blog on Creational Design Patterns](#)
 - [YT Video on Creational Design Patterns](#)

► Structural Design Patterns

- Deal with object composition, ensuring that objects work together in a system.
 - **Examples:** Adapter, Composite, Proxy.
 - Resources:
 - [Blog on Structural Design Patterns](#)
 - [YT Video on Creational Design Patterns](#)

► Behavioral Design Patterns

- Focus on communication between objects, making it easier to assign responsibilities between them.
 - **Examples:** Observer, Strategy, Command.
 - Resources:

- [Blog on Behavioral Design Patterns](#)
- [YT Video on Behavioral Design Patterns](#)

Week 3: Easy and Medium Design Problems

This week is dedicated to solving real-world design problems to reinforce your understanding of design principles and patterns. By working through easy to medium-level problems, you'll practice designing efficient systems that align with business requirements. Each problem will challenge your ability to structure class relationships, handle data, and apply patterns where needed.

- **Goal:** Solve practical design problems by applying LLD concepts.
- **Key Focus:** Object-oriented design, class diagrams, and pattern application in real-world scenarios.

► Design Problems (Easy)

1. [Parking Lot](#)
2. [Vending Machine](#)
3. [Stack Overflow](#)
4. [Logging Framework](#)
5. [Coffee Vending Machine](#)
6. [Traffic Signal](#)
7. [Task Management System](#)
8. [Pub-Sub System](#)
9. [Tic Tac Toe](#)
10. [Car Rental System](#)
11. [ATM Machine](#)
12. [Hotel Management System](#)
13. [Design a Bar Graph Library](#)
14. [Vending Machine](#)
15. [Inventory Management System](#)
16. [Tic Tac Toe](#)
17. [Design a Basic Address Book](#)
18. [Design a Coffee Machine](#)
19. [Task Scheduler](#)
20. [Design a Simple File System](#)

► Design Problems (Medium)

1. [LinkedIn](#)
2. [Social Networking Service](#)
3. [Elevator System](#)
4. [Library Management System](#)
5. [Restaurant Management System](#)
6. [Airline Management System](#)
7. [Digital Wallet System](#)
8. [Online Auction System](#)
9. [Concert Ticketing System](#)
10. [LRU Cache](#)
11. [Event Calendar System](#)
12. [Design a Ride-Sharing System](#)
13. [Library Catalog System](#)
14. [Online Food Delivery System](#)
15. [Design a Chat Application](#)
16. [Design a LinkedIn Clone](#)
17. [Design a Restaurant Management System](#)
18. [Design an Airline Management System](#)
19. [Design a Twitter Clone](#)
20. [Design a Hotel Management System](#)

Week 4: Hard Design Problems

In the final week, you will tackle complex and challenging low-level design problems. These hard design problems will push you to think critically about scalability, performance, and system efficiency. You will be required to apply your knowledge of design principles, patterns, and real-world system architecture. This week's problems will simulate real-world scenarios that demand detailed and well-thought-out solutions, preparing you for advanced low-level design interviews.

- **Goal:** Solve complex system design problems that require deep understanding of LLD concepts.
- **Key Focus:** Designing scalable, efficient, and maintainable systems, handling edge cases, and optimizing performance.

► Design Problems (Hard):

1. [Movie Ticket Booking System](#)
2. [Splitwise](#)
3. [Snake and Ladder](#)
4. [Online Shopping Service](#)
5. [Online Stock Brokerage System](#)
6. [Cricinfo](#)
7. [Chess Game](#)
8. [Ride Sharing Service](#)
9. [Food Delivery Service](#)
10. [Music Streaming Service](#)
11. [Course Registration System](#)
12. [Truecaller System](#)
13. [Inventory Management System](#)
14. [Ride Sharing System](#)
15. [Movie Ticket Booking System](#)
16. [Amazon Machine Design](#)
17. [Online Food Ordering and Delivery System](#)
18. [Car Rental System](#)
19. [Music Recognition System](#)
20. [Cricinfo](#)

Bonus Resources

Books:

- [Head First Design Patterns by Eric Freeman, Ph.D.](#)
- [Head First Object-Oriented Analysis & Design by Brett McLaughlin](#)
- [Clean Code by Robert C. Martin \(Uncle Bob\)](#)
- [Clean Architecture by Robert Martin](#)
- [Refactoring: Improving the Design of Existing Code by Martin Fowler](#)
- [Patterns of Enterprise Application Architecture by Martin Fowler](#)
- [Design Patterns: Elements of Reusable Object-Oriented Software](#)

Blogs:

- [Level Up Coding - Mastering Low-Level Design Technical Interviews](#)
- [InterviewBit - Low-Level Design Interview Questions by Anshuman Singh, Abhimanyu Saxena](#)
- [Sandeep Kumar - Low-Level Design Roadmap](#)
- [Educative - Low-Level & Advanced System Design by Fahim ul Haq](#)
- [Shankhya Chatterjee - Mastering Low-Level Design](#)
- [Animesh Gaitonde - How to Ace the Low-Level Design Interview](#)

- [freeCodeCamp - Design Patterns All Developers Need to Know by Quincy Larson](#)

Websites:

- [Refactoring.Guru - Design Patterns](#)
- [Alexander Shvets - SourceMaking Design Patterns](#)
- [GeeksforGeeks - Software Design Patterns by Sandeep Jain](#)

Code Repositories:

- [Kamran Ahmed - Design Patterns for Humans](#)
- [Prasad Gujar - Low-Level Design Primer](#)
- [Udit Agarwal](#)
- [InterviewReady - Low-Level Design by Gaurav Sen](#)
- [Kumaran Gowthaman](#)
- [Dov Amir - Awesome Design Patterns](#)